Package 'levygof'

October 22, 2026

Title Goodness-of-Fit Tests for Levy Degradation Processes

Version 0.0.0.9000

Description Goodness-of-fit tests for parametric Levy degradation processes (gamma, Wiener, inverse Gaussian, variance-gamma and Meixner processes) from one or several trajectories. With regular inspections the observed increments are tested; with irregular inspections the parameters are estimated by maximum likelihood (package 'levyfit'), the trajectories are resampled on a regular grid by stochastic bridges (package 'levyBridge', Grall-Maes (2012) <doi:10.1177/1748006X12462522>) and the reconstructed increments are tested with tests for independent and identically distributed samples.

License GPL-3

URL https://github.com/rytracetito/levy_packages/tree/main/levygof, https://github.com/rytracetito/levy_packages

BugReports https://github.com/rytracetito/levy_packages/issues

Encoding UTF-8

Language en

Roxygen list(markdown = TRUE)

RoxygenNote 7.3.3

Imports fBasics, gofgamma, gofIG, goft, gsl, levyBridge, levyfit, nortest, Runuran, stats, tseries

Suggests levysim, statmod, testthat (>= 3.0.0)

Remotes github::rytracetito/levy_packages/levyfit, github::rytracetito/levy_packages/levyBridge, github::rytracetito/levy_packages/levysim

Config/testthat/edition 3

Author Amedée Rytrace Tito [aut, cre], Christian Paroissin [aut]

Maintainer Amedée Rytrace Tito <rytracetito5@gmail.com>

Contents

gof.levy 2
tests_disponibles 5
test_adequation 7

Index 9

`gof.levy` *Goodness-of-fit test of a Lévy degradation process*

Description

Tests whether one or several degradation trajectories come from a given Lévy process (gamma, Wiener, inverse Gaussian, variance-gamma or Meixner), with unknown parameters. With regular inspections the observed increments are tested; with irregular inspections the parameters are estimated, the trajectories are resampled on a regular grid by stochastic bridges, and the reconstructed increments are tested.

Usage

```
gof.levy(
  data,
  model,
  test = NULL,
  method = "Bridge",
  option = 1,
  alpha = 0.05,
  B = 200
)
```

Arguments

`data`	The observed trajectories, in one of the formats of the packages `levyfit` and `levyBridge`: • one trajectory: a data frame whose first column contains the inspection times and second column the degradation levels; • several trajectories, each with its own inspection times: a list of such data frames; • several trajectories with the same inspection times: a matrix whose first column contains the times and the other columns the levels, for instance the result of a simulation function of `levysim`. The columns are read by position, not by name. The inspection times of a trajectory must be strictly increasing; they do not have to start at 0 (the first inspection is taken as origin).
`model`	`"HGP"` (gamma process), `"HWP"` (Wiener process), `"InvGaus"` (inverse Gaussian process), `"VG"` (variance-gamma process) or `"Meixner"` (Meixner process). Upper or lower case does not matter.
`test`	Names of the tests to apply; `NULL` (default) applies all the tests of the model, see `tests_disponibles()`.
`method`	Resampling method of the bridges, used with irregular inspections only (see `levyBridge::gamma_bridge()`): • `"Bridge"` (default, Grall-Maës, 2012): one regular grid per trajectory, with as many points as observations; • `"ExtBridge"`: regular grid containing all the inspection times, with a time step common to all the trajectories;

- `"MultiBridge"`: one regular grid common to all the trajectories, of step (total observed duration) / (total number of increments), with about as many points as observations.

`option` Used only with `method = "MultiBridge"`: `1` (default) to resample each trajectory up to its own last inspection, `2` to resample all the trajectories up to the last inspection time over all of them, the shorter ones being extended by increments of the process drawn with the estimated parameters (these values do not come from the data and lower the power; option 1 is recommended).

`alpha` Level of the tests, strictly between 0 and 1.

`B` Number of bootstrap replications of the tests that use a parametric bootstrap.

Value

An object of class `"gof.levy"`, printed by `print()`: a list with

- `model`, `method` (`NA` when no bridge was used), `alpha`, `B`;
- `regular`: `TRUE` when all the trajectories are inspected with a constant time step;
- `out_of_support`: `TRUE` when an observed increment is negative or zero under an increasing model (gamma, inverse Gaussian);
- `parameters`: maximum likelihood estimates of the parameters of the process (`NULL` when no bridge was used);
- `steps`: time step of the increments of each trajectory;
- `bridge`: the resampled trajectories (list of data frames with columns `t` and `X`, on the time scale of the data), or `NULL`;
- `increments`: data frame of the tested increments, with columns `id` (trajectory), `step` and `dx`;
- `results`: data frame of the tests, with a column `group` (`"all"` for the test on all the trajectories together, or the number of the trajectory) followed by the columns of `test_adequation()`.

Method

The null hypothesis H0 is: the trajectories are independent trajectories of the Lévy process `model`, with unknown parameters.

1. **Regular inspections** (each trajectory inspected with a constant time step Δ): the increments are i.i.d. with the law of X_Δ; they are tested directly.
2. **Irregular inspections**: the parameters are estimated by maximum likelihood on all the trajectories (package `levyfit`), then each trajectory is resampled on a regular grid by exact simulation of the bridge of the process with these parameters (package `levyBridge`, method `method`). Under H0 the reconstructed increments are i.i.d. with the law of $X_{t_{eq}}$, where t_{eq} is the time step of the grid; they are tested.
3. **Gamma and inverse Gaussian models**: these processes are increasing. A negative or zero observed increment is impossible under H0, which is then rejected for sure (p-value 0), without estimation nor bridge.

The tests are tests for i.i.d. samples with estimated parameters, see `test_adequation()` and `tests_disponibles()`.

Several trajectories

Each trajectory is tested alone (group 1, 2, ...). When all the trajectories have the same time step, their increments have the same law under H0: they form one i.i.d. sample and are also tested together (group `"all"`). This is always the case with the methods `"ExtBridge"` (step common to all the trajectories, the largest one such that every inspection time of every trajectory is a grid point) and `"MultiBridge"` (one grid common to all the trajectories). With the method `"Bridge"` the step t_N/N of each trajectory is generally different, and the increments of different trajectories are then not tested together: use `"MultiBridge"` or `"ExtBridge"` for one test on all the trajectories.

Randomness

The bridges and the bootstrap are random: use `set.seed()` before `gof.levy()` to reproduce a result.

References

Grall-Maës, E. (2012). Use of the Kolmogorov-Smirnov test for gamma process. *Proceedings of the Institution of Mechanical Engineers, Part O: Journal of Risk and Reliability*, 226(6), 624-634. doi:10.1177/1748006X12462522

See Also

`test_adequation()` for an i.i.d. sample of increments, `tests_disponibles()` for the list of the tests, `levyfit::gamma_mle()` for the estimation, `levyBridge::gamma_bridge()` for the bridges.

Examples

```
set.seed(1)

# 1. One gamma trajectory (a = 2, b = 3) inspected every 2 time units:
#    the observed increments are tested directly
t <- seq(0, 60, by = 2)
x <- c(0, cumsum(rgamma(30, shape = 2 * 2, rate = 3)))
gof.levy(data.frame(t, x), "HGP", B = 100)

# 2. Irregular inspections (a = 10, b = 5): estimation, gamma bridge, test
t2 <- c(0, cumsum(round(runif(30, 0.5, 3.5), 1)))
x2 <- c(0, cumsum(rgamma(30, shape = 10 * diff(t2), rate = 5)))
res <- gof.levy(data.frame(t2, x2), "HGP", test = c("KS", "AD"), B = 100)
res
res$parameters
res$results

# 3. The same trajectory with the method "ExtBridge"
gof.levy(data.frame(t2, x2), "HGP", test = c("KS", "AD"), method = "ExtBridge", B = 100)

# 3b. A small shape (a = 2): the reconstructed increments can be tiny but stay positive
x4 <- c(0, cumsum(rgamma(30, shape = 2 * diff(t2), rate = 3)))
gof.levy(data.frame(t2, x4), "HGP", test = "AD", method = "ExtBridge", B = 100)

# 4. Wiener model on the same data, at level 10 %
gof.levy(data.frame(t2, x2), "hwp", alpha = 0.1)

# 5. A decreasing step: the gamma model is rejected for sure
x3 <- x2
```

```
x3[10] <- x3[9] - 0.1
gof.levy(data.frame(t2, x3), "HGP", test = "KS")

# 6. Several trajectories with their own inspection times (a list):
#    one test per trajectory
liste <- lapply(1:3, function(k) {
  tk <- c(0, cumsum(round(runif(20, 0.5, 3.5), 1)))
  data.frame(t = tk, X = c(0, cumsum(rnorm(20, 0.5 * diff(tk), 1.5 * sqrt(diff(tk))))))
})
gof.levy(liste, "HWP", test = c("SW", "AD"))

# 6b. The same trajectories with "ExtBridge": common step, so also one test on all
gof.levy(liste, "HWP", test = c("SW", "AD"), method = "ExtBridge")

# 6c. "MultiBridge": one common grid of Grall-Maes type, also one test on all
gof.levy(liste, "HWP", test = c("SW", "AD"), method = "MultiBridge")
gof.levy(liste, "HWP", test = c("SW", "AD"), method = "MultiBridge", option = 2)

# 7. Several trajectories with the same irregular inspection times (matrix):
#    the grids have the same step, so the increments are also tested together
m <- cbind(t = t2, sapply(1:4, function(k) c(0, cumsum(rgamma(30, 10 * diff(t2), 5)))))
gof.levy(m, "HGP", test = "AD", B = 100)


# 8. Inverse Gaussian and variance-gamma models (slower)
ig <- c(0, cumsum(statmod::rinvgauss(30, mean = diff(t2), shape = 3 * diff(t2)^2)))
gof.levy(data.frame(t2, ig), "InvGaus", test = c("KS", "AD", "HK2"), B = 100)

g <- rgamma(30, shape = diff(t2) / 0.2, scale = 0.2)
vg <- c(0, cumsum(0.5 * g + 1.5 * sqrt(g) * rnorm(30)))
gof.levy(data.frame(t2, vg), "VG", test = "AD", B = 50)
```

`tests_disponibles` *Available goodness-of-fit tests*

Description

Lists the tests that `gof.levy()` and `test_adequation()` can apply to each model.

Usage

```
tests_disponibles(model = NULL)
```

Arguments

`model`	`NULL` (default) for all the models, or one model: `"HGP"` (gamma process), `"HWP"` (Wiener process), `"InvGaus"` (inverse Gaussian process), `"VG"` (variance-gamma process) or `"Meixner"` (Meixner process). Upper or lower case does not matter.

Value

With `model = NULL`, a named list with one vector of test names per model; otherwise the vector of test names of this model.

The tests

All the tests are tests of a composite hypothesis: the increments $\Delta x_1, \ldots, \Delta x_n$ are i.i.d. with a law of the family of the model, whose parameters are unknown and estimated on the sample.

Gamma process (`"HGP"`), increments $\Gamma(a\Delta, b)$:

- `"HME1"`: Henze, Meintanis and Ebner (2012), Laplace transform (`gofgamma::test.HME1`, tuning parameter 1);
- `"BE"`: Betsch and Ebner (2019), characterization by a fixed point (`gofgamma::test.BE`, tuning parameter 1);
- `"VGE"`: Villasenor and Gonzalez-Estrada (2015), ratio of two variance estimators (`goft::gamma_test`), with a p-value by parametric bootstrap;
- `"KS"`, `"CvM"`, `"AD"`, `"Watson"`: Kolmogorov-Smirnov, Cramer-von Mises, Anderson-Darling and Watson (`gofgamma`).

The tests of `gofgamma` give the decision by comparison with a bootstrap critical value, without p-value.

Wiener process (`"HWP"`), increments $N(\mu\Delta, \sigma^2\Delta)$: tests of normality with estimated mean and variance.

- `"AD"`, `"CvM"`, `"Lilliefors"`: Anderson-Darling, Cramer-von Mises and Lilliefors (Kolmogorov-Smirnov) tests (`nortest`);
- `"SW"`: Shapiro-Wilk (`stats::shapiro.test`);
- `"JB"`: Jarque-Bera (`tseries::jarque.bera.test`);
- `"DAgostinoPearson"`: D'Agostino-Pearson omnibus test K^2, combining skewness and kurtosis (`fBasics::dagoTest`).

Inverse Gaussian process (`"InvGaus"`), increments $IG(\mu\Delta, \eta\Delta^2)$:

- `"ABEV1"`, `"ABEV2"`: Allison, Betsch, Ebner and Visagie (2022) (`gofIG`, tuning parameter 10, maximum likelihood);
- `"HK1"`, `"HK2"`: Henze and Klar (2002) (`gofIG`);
- `"KS"`, `"CvM"`, `"AD"`: Kolmogorov-Smirnov, Cramer-von Mises and Anderson-Darling (`gofIG`);
- `"VGE_ratio"`, `"VGE_gamma"`: Villasenor, Gonzalez-Estrada and Ochoa (2019), ratio of variances and transformation to gamma variables (`goft::ig_test`).

Variance-gamma (`"VG"`) **and Meixner** (`"Meixner"`) **processes**: `"KS"`, `"CvM"`, `"AD"`, `"Watson"`. The parameters are estimated by maximum likelihood (`levyfit::vg_mle()`, `levyfit::meixner_mle()`), the distribution function is computed by numerical integration of the density, and the p-value is obtained by parametric bootstrap.

See Also

`gof.levy()`, `test_adequation()`

Examples

```
# All the models and their tests
tests_disponibles()

# The tests of one model (upper or lower case)
tests_disponibles("HGP")
tests_disponibles("meixner")
```

`test_adequation` *Goodness-of-fit tests for an i.i.d. sample of increments*

Description

Applies goodness-of-fit tests to a sample of increments of a Lévy process observed with a constant time step: the increments are then independent and identically distributed. This is the last step of `gof.levy()`, also useful alone when the increments are already available (regular inspections, or increments reconstructed by the package `levyBridge`).

Usage

```
test_adequation(dx, model, test = NULL, alpha = 0.05, B = 200)
```

Arguments

`dx`	Numeric vector of at least 3 increments, all over the same time step.
`model`	`"HGP"` (gamma process), `"HWP"` (Wiener process), `"InvGaus"` (inverse Gaussian process), `"VG"` (variance-gamma process) or `"Meixner"` (Meixner process). Upper or lower case does not matter.
`test`	Names of the tests to apply; `NULL` (default) applies all the tests of the model, see `tests_disponibles()`.
`alpha`	Level of the tests, strictly between 0 and 1.
`B`	Number of bootstrap replications, for the tests whose p-value or critical value is computed by parametric bootstrap.

Details

The null hypothesis is: the increments are i.i.d. with a law of the family of the model (parameters unknown). Under the gamma and inverse Gaussian models the increments are positive with probability one: a negative or zero increment is impossible under H0, and all the tests then reject H0 with a p-value 0, without computation.

A test that fails (for instance a sample too small for the estimation) does not stop the others: its row has the decision `"not computed"` and the error message.

The tests are described in `tests_disponibles()`. The bootstrap tests are random: use `set.seed()` to reproduce a result.

Value

A data frame with one row per test and the columns:

- `test`: name of the test;
- `statistic`: value of the test statistic;
- `p.value`: p-value (`NA` for the tests of `gofgamma`, which give a critical value instead);
- `critical.value`: bootstrap critical value (tests of `gofgamma` only);
- `decision`: `"reject H0"`, `"do not reject H0"`, or `"not computed"` when the test failed;
- `message`: reason of a failure or of a certain rejection, `""` otherwise.

See Also

gof.levy() for a trajectory or several trajectories, tests_disponibles() for the list of the tests.

Examples

```
set.seed(1)

# 1. Gamma increments (time step 2, a = 2, b = 3): all the tests of the model
dx <- rgamma(40, shape = 2 * 2, rate = 3)
test_adequation(dx, "HGP", B = 100)

# 2. Some tests only, at level 10 %
test_adequation(dx, "HGP", test = c("KS", "AD"), alpha = 0.1, B = 100)

# 3. Normal increments: tests of the Wiener model
test_adequation(rnorm(50, mean = 1, sd = 2), "HWP")

# 4. The same normal increments are negative: the gamma model is rejected for sure
test_adequation(rnorm(50), "HGP", test = c("KS", "AD"))

# 5. Inverse Gaussian increments
dx_ig <- 1 / rgamma(40, 3) # not inverse Gaussian
test_adequation(dx_ig, "invgaus", test = c("KS", "AD", "VGE_ratio"), B = 100)


# 6. Variance-gamma model (slower: estimation and numerical integration
#    in each bootstrap replication)
g <- rgamma(60, shape = 1 / 0.5, scale = 0.5)
dx_vg <- 0.5 * g + 1.5 * sqrt(g) * rnorm(60)
test_adequation(dx_vg, "VG", test = "AD", B = 50)
```

Index

gof.levy, 2
gof.levy(), 5–8

levyBridge::gamma_bridge(), 2, 4
levyfit::gamma_mle(), 4
levyfit::meixner_mle(), 6
levyfit::vg_mle(), 6

set.seed(), 4, 7

test_adequation, 7
test_adequation(), 3–6
tests_disponibles, 5
tests_disponibles(), 2–4, 7, 8